

Structurally Valid, Semantically Wrong: Measuring the Validation Gap for AI-Generated Web Inputs

Zahid Rahimoon

Independent Researcher

Email: zahidrahimoon22@gmail.com

Abstract—A web application’s schema validation checks that an input is well formed. It does not check the business rules that decide whether the input is appropriate, and in the application studied here most of those rules are not enforced in code. This paper measures how often inputs that pass validation still break such rules, and whether inputs written by large language models (LLMs) differ from inputs produced in other ways. A Next.js testbed with an enumerated reference of 43 business rules received 3881 inputs through its real HTTP validation path. The inputs came from seven generation conditions: two LLM conditions, random values, rule-based adversarial values, boundary values, and two reference conditions. Semantic validity was decided by rule functions, by an independent judge model and by a human annotation of 200 inputs. Of the decided, structurally valid LLM inputs, 30.9 % (95 % CI 28.2–33.7) broke at least one rule. The rate was 20.3 % when the model was asked only for legitimate values. The gap is not specific to AI. Random structurally valid inputs broke rules more often (53.3 %), and giving the model the field’s purpose and its rules lowered the odds of a violation by 77 %. Five candidate validation layers were then evaluated. None reached the pre-registered target of 70 % recall at a 5 % false-positive rate. The best fast design detected 27.1 %, and most of what it caught were violations of rules the application could enforce in code. On a four-core CPU the rule layer added at most 14 ms to p97.5 latency and the embedding layer 65–96 ms for a single client. At the median the small language model added about half a second and the LLM judge 1.6–7.7 s, and the judge saturated on two of three endpoints at eight concurrent clients. The study covers one application and CPU-only open models. The judge agreed with a human annotator at Cohen’s $\kappa = 0.24$, so results that depend on judgement rules are exploratory.

Index Terms—input validation, business logic, large language models, web applications, software testing, empirical study

I. INTRODUCTION

A web application accepts an input only after it passes validation. In the JavaScript and TypeScript ecosystem that validation is usually a schema. The schema is a declarative object that states the type, presence, length, pattern, numeric range, enumeration and format of every field. It is checked on the client, again on the server, and finally by database constraints. Schemas of this kind are well understood [1], [2] and have been studied empirically at scale [3]. Other rules depend on what the application is for: a promotional code has to apply to the item being bought, a workshop cannot start in the past, and a review is supposed to be about the workshop it is attached to. Some of these can be written as custom refinements, although in practice most are never written down

anywhere. The distinction that matters is therefore between the business rules an application enforces and the many it leaves unenforced.

That limitation is not new. Felmetzger et al. separated input-validation flaws, which admit a general specification, from logic vulnerabilities, which do not [4]. Later work detected such flaws from behavioural traces [5], from client-server validation mismatches [6] and from over-permissive API payloads [7]. What has changed since then is who writes the input. Large language models (LLMs) now fill forms, operate web applications on a user’s behalf, and generate test data [8]–[10].

Recent work shows that LLM output which satisfies a schema is often wrong in meaning. Schema-constrained ordering agents reach full schema validity while semantic success falls as low as a few per cent [11]. Forcing a small model into a schema raised validity while executable accuracy fell in a calendar tool-call task [12]. Across 21 models, schema compliance exceeds value accuracy by 15 to 25 percentage points [13]. Schema-first tool interfaces reduce interface misuse and increase schema-valid semantic misuse [14]. In one enterprise platform the most structurally successful model had among the lowest semantic satisfaction [15]. All of this concerns model outputs that a tool, an API or a renderer consumes. Two questions about the input boundary of a web application remain open, and there the accepted value is stored and later shown to other users or to an assistant. How often do LLM-written inputs that pass validation break the application’s business rules? And is that rate different from the rate for other producers of structurally valid input?

Web testing studies cannot answer either question, because they treat acceptance by the application as validity [8], [9]. One such study notes that some invalid data, such as incomplete e-mail addresses, may be submitted successfully because the application lacks the corresponding check [8]. A robustness study that uses status codes as its oracle cannot see failures that return success [16]. Where the semantic validity of generated API values is measured, a base eight-billion-parameter model produces domain-coherent values under a quarter of the time [17], but that study has no non-LLM comparison.

This paper measures the gap at the input boundary of one purpose-built application that has an explicit rule reference. It compares LLM inputs with random, rule-based, boundary and reference inputs on the same outcome. The contributions are:

- **C1.** A measurement of how often structurally valid inputs break unenforced business rules, for LLM and non-LLM producers alike. The rate is broken down by prompt intent, field category and target, and a sensitivity analysis is reported for every design choice that affects it.
- **C2.** A controlled comparison of seven input-generation conditions on one outcome. The comparison shows that the gap is not specific to AI-generated input. Random structurally valid inputs exceed LLM inputs, and field context reduces LLM violations.
- **C3.** An evaluation of five validation layers (rules, embedding similarity, a small language model, a hybrid, and an LLM judge) at a bounded false-positive rate on held-out fields. None of them reaches the pre-registered target.
- **C4.** The request-path cost of those layers on commodity CPU hardware. The guardrail literature I reviewed reports no comparable figures [18]–[21].
- **C5.** The testbed, harness, rule reference, raw data and analysis scripts, together with the defects found during execution and the way each one was handled.

Section II fixes the terminology and Section III positions the work. Section IV states the research questions and hypotheses, which were frozen before data collection. Section V describes the method and Section VI the validation layers. Section VII reports the results and Section VIII interprets them. Sections IX and X give the threats to validity and the conclusions.

II. BACKGROUND AND TERMINOLOGY

Structural validation is every check the application performs before business logic: the runtime schema (type, presence, length, pattern, range, enumeration, format, and any refinements written into it) together with service-level checks and database constraints. An input *passes* structural validation when the application accepts it.

Semantic validity is whether a structurally valid value is appropriate for the application’s purpose: consistency across fields, domain plausibility, compliance with business rules, and safety for downstream consumers such as an LLM component. It is operationalised as satisfying every rule in a frozen, enumerated reference.

An input that passes structural validation and violates at least one rule is **SV-SI** (structurally valid, semantically invalid). The *conditional* SV-SI rate is the share of SV-SI inputs among the structurally valid inputs whose rules were all decided. It is the primary outcome. A generator that produces more well-formed values would otherwise look worse merely by passing more often.

Terminology differs across communities. The OWASP Input Validation Cheat Sheet separates syntactic from semantic validation [22]. I count a range that the developer wrote into the schema as structural, because the application enforces it. Security research speaks of logic vulnerabilities [4], [5] or of business logic vulnerabilities [23]. API testing research speaks of inter-parameter dependencies [24], [25], and one older paper uses “semantic-based user input validation” in the sense used

here [26]. In LLM security the same adjective usually means adversarial intent [27]. My usage concerns whether the data is appropriate, which includes that case.

III. RELATED WORK

A. Limits of structural validation

JSON Schema has a formal semantics and a known complexity [1], [2], and empirical studies describe which constraints practitioners encode [3]. Inter-parameter dependencies of the form *requires*, *or*, *only-one*, *all-or-none*, *zero-or-one* and arithmetic relations cannot be stated in an OpenAPI document [24], [25]. That gap has motivated generating server-side validation code from a separate dependency language [28]. Schema-derived fuzzers inherit the same boundary [29], and surveys of input-validation vulnerabilities remain organised around injection classes [30].

B. Logic flaws

Waler inferred likely invariants over session state and database query parameters and model-checked them, which uncovered previously unknown logic vulnerabilities [4]. Related classes are detected by black-box behavioural patterns [5], by parameter-tampering analysis [6], by mass-assignment testing [7] and by excessive-data-exposure fuzzing [31]. Semantic anomaly detection in this invariant sense predates the web literature [32]. Recent work applies LLMs to find business-logic vulnerabilities in code [23], [33], surveyed among other security uses [34]. In the studies I reviewed the inputs are crafted by an attacker or a tester, and they are not compared across producers.

C. LLM-generated inputs for forms and APIs

LLMs are better than random and type-based generators at producing inputs that applications accept. On thirty real forms an LLM-driven approach reaches a form passing rate of 83 %, against 23 % for a static type-based generator and 3 % for random crawling [9]. Across 146 forms and eleven models, the best model submits over 98 % of its generated inputs [8]. The oracle in both cases is acceptance by the application. Where semantic validity is measured separately, a base model reaches 22.9 % domain-coherent values, against 72.4 % after fine-tuning [17]. Related work mines input constraints from specifications [35], [36], studies text input for mobile interfaces [37], benchmarks form-filling agents [10], and measures the schema validity of synthetic records [38]. I found no study that compares LLM inputs with random, rule-based and reference inputs on a semantic outcome.

D. Schema-valid but semantically wrong outputs

For model outputs the pattern is documented by several 2026 studies. OrderBench separates syntactic validity, schema validity and semantic success [11], while the constraint-tax study measures how much accuracy is lost when hard schema decoding replaces prompting [12]. On a multi-source benchmark the JSON pass rate stays ahead of value accuracy [13], and in a controlled tool-API comparison schemas shift failures

from interface misuse to semantic misuse [14]. The pattern also reaches production: in one enterprise platform, workflows passed validation and then could not execute [15]. Taxonomies of structured-output error [39], cross-column constraint repair [40] and neuro-symbolic verification [41] develop the theme. Tool-use benchmarks evaluate the calls an LLM emits against a task’s expected answer, either under realistic multi-turn use [42] or with a focus on filling parameter values correctly [43]. In all of these the semantic check is the oracle rather than a layer that is itself evaluated, and the setting is model output rather than application input.

E. Data validation beyond schemas

Data-management research validates data beyond structural schemas. Examples are anomaly detection on data fed to machine-learning pipelines [44], declarative constraint checks that act as unit tests for data [45], and learned detection of the semantic types of columns [46]. These systems work on datasets or columns in batch. I study per-request validation of one input against an application’s business rules. Classifying a single field entry as valid or invalid from a context embedding and statistics of recent entries has been described in a patent [47]. My EMB design is a simpler, untrained relative of that idea, and I evaluate it empirically.

F. Semantically inappropriate input as a security problem

Indirect prompt injection established that models do not separate data from instructions [48], that attacks on deployed applications succeed [49], [50], and that surface quality checks do not distinguish compromised text from clean text [51]. An attacker can insert prompt fragments through an ordinary form field and later drive a database write with them [52]. The pattern extends to classic web sinks [53] and occurs in the wild [54]. Benchmarks [55]–[57] and a journal study [58] extend the coverage. Inputs in this line of work are attacker-crafted and mostly free text.

G. Detection layers and their cost

Input-side detectors include fine-tuned classifiers [18], [59], embedding classifiers [19], [60], small transformers [61], [62], intrinsic-feature probes [63], [64] and LLM judges [65], [66]. What constrains deployment is the false-positive rate on benign traffic [67]. Cost evidence, by contrast, is thin and tied to GPUs. A detector presented as deployable gives no timing at all [18], whereas an embedding guardrail puts its embedding floor at 35 ms on a datacentre GPU [19] and a layered framework reports a 61 ms median on an A100 with model inference excluded [20]. Surveys of the area note that cost and utility are routinely overlooked [21], [68], [69]. Only one study compares regex, classifier and LLM tiers head to head, and it does not state its hardware [70]. LLM judges, meanwhile, can themselves be redirected by the content they assess [71]. I found no measurement of a validator for data appropriateness on structured request payloads on CPU.

H. Position of this work

Three lines of work come closest among the studies I reviewed, and the design of this study responds to a specific difference in each. The output-side studies measure schema-valid but wrong values that a model emits [11]–[13], [15], where the oracle is the task’s expected answer. I measure values that an application accepts and stores, where the application raises no error and the value is later read by people or by an assistant. The form and API testing studies generate inputs with an LLM and count acceptance by the application [8], [9], so an accepted but inappropriate value counts as a success; one study measures domain coherence separately but has no non-LLM comparison [17]. I separate the two outcomes and label semantic validity against a frozen rule reference, independently of the application. The guardrail studies evaluate detectors for adversarial intent, and where they report cost at all it is on datacentre GPUs [19], [20]. I evaluate detectors for ordinary data appropriateness, on structured payloads, on a four-core CPU, and I report detection quality and cost together. The comparison across seven producers, including non-LLM ones, is what lets me ask whether the gap belongs to AI writers or to the validation boundary itself.

IV. RESEARCH QUESTIONS AND HYPOTHESES

The following were frozen, with the primary metrics and the analysis plan, before any experimental input was generated. Prompt family P5 (injection, Section V) is excluded from all primary rates and analysed separately.

- **RQ1.** Do LLM-generated inputs pass structural validation while violating business rules? *H1:* the conditional SV-SI rate of the LLM conditions has a 95 % Wilson interval whose lower bound exceeds 0.05.
- **RQ2.** How does this vary across field categories? *H2:* rates differ across categories; free text and cross-field highest, simple scalar and structured (enumeration) lowest.
- **RQ3.** How do LLM inputs compare with other producers? *H3a:* LLM with field context exceeds random; *H3b:* LLM with field context exceeds benign-unusual reference inputs; *H3c:* LLM without field context exceeds random.
- **RQ4.** Which violation types dominate? Exploratory.
- **RQ5.** Can an added semantic layer detect SV-SI inputs? *H5:* at least one non-judge design reaches recall ≥ 0.70 at a false-positive rate (FPR) ≤ 0.05 on held-out fields.
- **RQ6.** What does it cost? *H6:* rule and embedding designs add at most 50 ms to p95 request latency while an LLM judge adds at least 1000 ms.

V. METHOD

A. Subject under study

The subject is a purpose-built Next.js 16 application written in TypeScript. It uses Zod 4 for validation on the client and the server, Prisma with SQLite for persistence, and a local Ollama runtime for two AI-facing features, support-ticket triage and a booking assistant. It has seven input surfaces and 39 fields.

A reference file enumerates 43 business rules. Five of them are enforced in code, two of those as Zod refinements (a birth date not in the future, and a workshop end after its start), and 38 are not enforced. That profile was chosen to resemble small production applications. Each rule is classified in advance as objective, meaning decidable from the input, the database and the clock (19 rules), or as judgement (24 rules). The application and the rule reference were tagged before any input was generated (commit 62a7267), and the commit is recorded with every run. Using a purpose-built subject is a deliberate trade. Real applications do not enumerate their business rules, and without a complete reference the semantic outcome has no ground truth.

B. Targets and generation conditions

Seventeen targets covering 22 fields were selected to represent every field category, every directly AI-facing field, and five cross-field pairs whose rule relates the two members. Each target carries a purpose description, its rule identifiers and its structural constraints. Seven conditions produce values for each target. **A** is valid reference values. **B** is random values that satisfy the schema, drawn from a seeded generator of word sequences, numbers and dates within the schema’s limits, with no attempt at meaning. **C** is rule-based adversarial values, one template per unenforced rule. **D** is an LLM given the field name and the structural constraints only. **E** is the same LLM given the field’s purpose and rules as well. **F** is boundary values at the schema edges. **G** is benign but unusual reference values. Conditions A and G were specified as human-written. In this execution their 272 values were first drafted with the help of a language model, before the main generation, and checked against the schemas and the objective rules. I then went through every value myself, kept 262, replaced 3 with values of my own, deleted 7, and confirmed that the kept values are ones I would have written. The application accepted 0 of the replacements, because all 3 were promotional codes it does not recognise. They are condition A’s only structural rejections, so the reviewed conditions rest on the kept values. I therefore call these conditions human-curated rather than human-written.

C. LLM generation

Conditions D and E use qwen3:4b (Q4_K_M) through Ollama on a four-core CPU, at temperature 0.8, nucleus 0.9, with a fixed seed per run, three runs, and five values per call. Seven prompt families are stored as hashed files: P1 legitimate values, P2 unusual values, P3 values that satisfy the constraints but “would be wrong for the application to accept”, P4 values whose meaning depends on downstream interpretation, P5 injection for AI-facing targets only, P6 boundary values, and P7 Unicode variants. Because P3 asks for violations, results are reported per family as well as pooled. The model identifier, digest, temperature, seed, prompt file and prompt hash are recorded for every value. Failed calls were counted and never regenerated. There were 88 failure records in 52 prompt cells, of which 75 were call errors such as calls aborted at the

120s timeout and 13 were unparseable replies. A further 430 generated values used key names other than the target’s. A harness defect had filled those keys from the form’s default values, so the records measure the defaults rather than the model, and they are treated as generation failures.

D. Structural measurement

Each input is measured twice. It is checked against the same schema object the application imports, and it is submitted over HTTP to the real route handler, which records the status, the service errors and the time. The structural outcome is the application’s own decision, so an input rejected by one of the five enforced business checks counts as a structural rejection. Booking state is reset to the seed before each booking request.

E. Semantic ground truth

Objective rules are decided by rule functions. The 11 objective rules that apply to the 17 targets all have one. Judgement rules are decided by a judge model from a different family than the generator (gemma3:4b, temperature 0). The judge sees the field purpose, the rule text and the submitted value of the target’s fields. It never sees the condition, the prompt family or the intent. The judge does not see the rest of the record. Seven judgement rules depend on other fields, such as capacity being plausible for the location, and those were judged without that context. A sensitivity analysis excludes them. A human annotator labelled 200 structurally valid inputs blind to condition, with 20 items repeated to estimate the annotator’s consistency. Where a human verdict exists it takes precedence over the judge. An input is *decided* when every applicable rule has a verdict. For compute reasons the judge was applied to a stratified sample: 1200 structurally valid inputs drawn round-robin across target and condition with a fixed seed, plus 136 inputs for two booking targets that were re-measured after a defect (Section IX), up to 14 per stratum and as many as were available.

F. Analysis population and statistics

Of the 3881 inputs, the following are excluded: 430 key-mismatch records, 131 records produced by inadvertent retries of failed calls, 74 injection-family inputs, and 10 reference values that the reviewer deleted or replaced. That leaves 3236 inputs. Of these, 2917 passed structural validation and 1876 are decided. The remaining passes were either not in the judge sample (1001) or received no usable verdict (40). Rates carry Wilson 95 % confidence intervals (CIs), and contrasts are reported as odds ratios (ORs). H1 is evaluated on the lower bound of its interval, H2 with a χ^2 test and Cramér’s V , and H3a to H3c with three one-sided Fisher tests under Holm correction. The judge sample gives the conditions different target mixes, so each contrast is also tested within target with the Cochran–Mantel–Haenszel (CMH) test, which was added after data collection. Values generated in one call, and targets within a category, are not independent, so the intervals are likely too narrow. Per-run and per-target rates are reported for that reason. Every other analysis is labelled exploratory. It was

fixed in advance that if judge–human agreement on judgement rules falls below $\kappa = 0.4$, every result that depends on those rules is reported as exploratory. Every number in this paper is generated from the raw data by scripts that are released with it.

VI. SEMANTIC VALIDATION LAYERS

Five designs run as a drop-in check after structural validation. **R** packages the objective rule functions. Those functions also define the ground truth for objective rules, so **R** cannot produce a false positive on them, and it serves as a floor rather than a competitor. **EMB** scores a value by its cosine distance from an embedding (nomic-embed-text) of the field’s purpose and rules. It uses no labelled examples, unlike the trained classifier the plan anticipated. **SLM** prompts a small language model (SLM; qwen3:1.7b) with the field purpose and the rules, and asks for a verdict and a confidence. **HYB** runs **R** first and then **EMB** on what **R** passes. **JUDGE** sends the same prompt to qwen3:4b, the generator’s model, and serves as a reference point. **SLM** and **JUDGE** therefore receive a rubric similar to the ground-truth judge’s. The prompted designs run at temperature 0, and a reply without a verdict counts as a pass, because the deployed layer fails open.

Evaluation is leave-fields-out over five folds. In each fold the threshold is the smallest score at which at most 5 % of the valid training-fold inputs are flagged, and that threshold is applied to the held-out fields. Designs are compared on recall at that operating point, together with FPR, precision and the threshold-free area under the receiver operating characteristic (ROC) curve (AUROC). The model-based designs were scored on a stratified sample of 575 decided inputs, of which 236 are SV-SI, and all designs are compared on that sample. The fast designs were also scored on all 1937 decided inputs, a population that includes the injection family and the retried records.

VII. RESULTS

A. Rate of semantic violations (RQ1)

Among decided, structurally valid LLM inputs, 338 of 1094 (30.9 %, 95 % CI 28.2–33.7) broke at least one business rule (Table I), so H1 is supported. The pooled rate includes prompt family P3, which asks for rule violations. Per family (Table II), the rate was 20.3 % for legitimate-value prompts and 43.3 % when violations were requested. Without P3 it was 28.7 %. Agreement between the judge and the human annotator is below the pre-registered threshold (Section VII E), so every rate that depends on judgement rules is exploratory, this one included. The objective-rule rate below depends only on the rule functions and is not affected. H1 holds under every sensitivity variant in Table III, including objective rules alone (7.9 %, CI 6.8–9.2), which do not depend on the judge. One target dominates. On workshop dates (F15+F16) nearly every model value was in the past, because the prompts never state the current date. Without that target the rate is 21.5 %. Per run, the rate was 31.1, 41.6 and 40.9 % without field context, and 25.7, 24.9 and 22.9 % with it. Including the retried records

gives 30.5 %. The reference conditions bound the labelling from both sides. Valid inputs were labelled invalid in 3.8 % of cases, and adversarial inputs in 69.1 %. About a third of the designed violations were therefore missed and a few valid inputs were flagged, so labelling error runs in both directions.

B. Field categories (RQ2)

Category is associated with the outcome ($\chi^2(6) = 184.5$, $p < 0.001$, Cramér’s $V = 0.31$, a moderate association), so H2’s test is supported (Table IV). These rates rest on judgement rules as well, so they are exploratory. The directional expectation is only partly met. Free text (51 %) and cross-field (45 %) are high, as expected. Structured fields are the highest (57 %) rather than among the lowest, and financial fields are the lowest (12 %). Categories contain between one and four targets (Table V). The structured category is a single target, a ticket’s priority and category, and the judge decided its rules without seeing the ticket text. The reversal should therefore not be read as a property of enumerated fields in general.

C. Comparison of producers (RQ3)

None of the three pre-registered contrasts is supported (Table VI), and two point the other way. They are computed on labels that include judgement rules, so under the agreement gate they are exploratory as well. The objective-rule sensitivity row shows the same direction without the judge. Random structurally valid inputs broke rules in 53.3 % of cases. That is more often than LLM inputs without field context (37.7 %) or with it (24.5 %), and it holds in every sensitivity variant (Table III). Within target (CMH, post hoc), the odds of a violation were lower for LLM inputs with context than for random inputs (odds ratio 0.16, 95 % CI 0.11–0.23), and lower without context (0.47, 0.34–0.66). Adversarial inputs exceeded random ones (2.18, 1.20–3.95). Field context lowered the odds relative to the schema alone (0.23, 0.15–0.35). LLM inputs with context and benign-unusual reference inputs did not differ detectably (1.32, 0.77–2.27, $p = 0.29$). The interval is wide, and the reference inputs are AI-drafted values curated by a human reviewer, so this compares model input with human-curated input rather than with input a person wrote unaided. Fig. 1 shows the rates by condition.

An exploratory variation repeated conditions D and E on ten targets with families P1 to P4, changing only the generator (Table VII). The 1.7-billion-parameter model produced more violations than the 4-billion model, both on objective rules (4.8 % against 1.1 %, odds ratio 4.4, $p_{\text{Holm}} = 0.001$) and on all rules (35.0 % against 22.3 %, $p_{\text{Holm}} = 0.02$). Neither the 8-billion model (23.1 %) nor temperature 0 (26.3 %) differed detectably. With one run per generator and a smaller judge sample, this only indicates that a smaller model may widen the gap.

D. Violation types (RQ4)

Counts of judgement-rule violations depend on the judge and are exploratory. Judgement-rule violations outnumber objective ones (502 against 272, across all conditions). The single

TABLE I

STRUCTURAL AND SEMANTIC OUTCOMES BY INPUT-GENERATION CONDITION. THE CONDITIONAL SV-SI RATE IS THE SHARE OF STRUCTURALLY VALID INPUTS THAT VIOLATE AT LEAST ONE BUSINESS RULE; IT IS THE PRIMARY OUTCOME, COMPUTED OVER *decided* PASSES: THOSE WHOSE EVERY APPLICABLE RULE RECEIVED A VERDICT (PASSES OUTSIDE THE JUDGE SAMPLE ARE UNDECIDED AND EXCLUDED). PROMPT FAMILY P5 (INJECTION) AND RETRIED RECORDS ARE EXCLUDED. INTERVALS ARE WILSON 95% CONFIDENCE INTERVALS. *SPECIFIED AS HUMAN-WRITTEN; VALUES AI-DRAFTED AND THEN REVIEWED VALUE BY VALUE BY THE RESEARCHER (HUMAN-CURATED).

Condition	n	Struct. pass (%)	Passes	Decided	SV-SI	Cond. SV-SI (%)	95% CI
A valid human*	134	97.8	131	130	5	3.8	[1.7, 8.7]
B random	425	93.6	398	362	193	53.3	[48.2, 58.4]
C rule-based adversarial	85	96.5	82	81	56	69.1	[58.4, 78.1]
D LLM context-free	1268	84.0	1065	530	200	37.7	[33.7, 41.9]
E LLM context-aware	1108	92.8	1028	564	138	24.5	[21.1, 28.2]
F boundary	85	96.5	82	81	26	32.1	[22.9, 42.9]
G benign-unusual human*	131	100.0	131	128	22	17.2	[11.6, 24.7]

TABLE II

CONDITIONAL SV-SI RATE OF MODEL-GENERATED INPUTS BY PROMPT FAMILY (DECIDED STRUCTURAL PASSES; P5 EXCLUDED). P3 EXPLICITLY ASKS THE MODEL FOR VALUES THE APPLICATION SHOULD REJECT; P1 ASKS FOR LEGITIMATE VALUES. D: WITHOUT FIELD CONTEXT; E: WITH PURPOSE AND RULES.

Prompt asks for	D (%)	E (%)	SV-SI	D+E (%)	95% CI
P1 legitimate	25.9	14.4	43/212	20.3	[15.4, 26.2]
P2 unusual	36.2	26.1	52/163	31.9	[25.2, 39.4]
P3 rule violations	48.1	39.1	71/164	43.3	[35.9, 50.9]
P4 context-dependent	29.1	20.6	47/193	24.4	[18.8, 30.9]
P6 boundary	46.7	22.3	56/169	33.1	[26.5, 40.5]
P7 Unicode variation	45.6	27.2	69/193	35.8	[29.3, 42.7]
All (H1)			338/1094	30.9	[28.2, 33.7]
All except P3			267/930	28.7	[25.9, 31.7]

TABLE III

SENSITIVITY OF THE CONDITIONAL SV-SI RATE TO DESIGN CHOICES. LLM: CONDITIONS D AND E POOLED; B: RANDOM; G: BENIGN-UNUSUAL (HUMAN-CURATED). "WITHOUT CONTEXT RULES" DROPS SEVEN JUDGEMENT RULES THE JUDGE HAD TO DECIDE WITHOUT THE REST OF THE RECORD. "OBJECTIVE RULES ONLY" USES NO JUDGE. TARGET-WEIGHTED AVERAGES PER-TARGET RATES BY EACH TARGET'S SHARE OF STRUCTURAL PASSES. P1 AND P3 APPLY TO MODEL CONDITIONS ONLY.

Variant	LLM (%)	95% CI	B (%)	G (%)
Pre-registered (all rules)	30.9	[28.2, 33.7]	53.3	17.2
P1 (legitimate) prompts only	20.3	[15.4, 26.2]	–	–
Without P3 (asks for violations)	28.7	[25.9, 31.7]	53.3	17.2
Without dates (F15+F16)	21.5	[19.0, 24.2]	50.4	17.5
Objective rules only	7.9	[6.8, 9.2]	10.3	0.8
Without context rules	24.5	[22.1, 27.2]	36.5	9.4
Target-weighted	30.2	–	53.3	17.2

most frequent violation is objective: a workshop must start in the future (146). It is followed by a profile bio addressed to the assistant (64), a ticket priority inconsistent with its text (61) and duration plausibility (57). The ticket rule was judged without the text it depends on. On AI-facing fields, 81 structurally valid LLM inputs violated a rule. Of those, 20 came from the injection family and 61 from the other prompt families.

TABLE IV

CONDITIONAL SV-SI RATE BY INPUT-FIELD CATEGORY, OVER DECIDED STRUCTURAL PASSES FROM ALL CONDITIONS. A χ^2 TEST OF INDEPENDENCE GIVES $\chi^2 = 184.5$, $p < 0.001$, CRAMÉR'S $V = 0.314$.

Field category	Decided	SV-SI	Cond. SV-SI (%)	95% CI
Structured	107	61	57.0	[47.5, 66.0]
Free text	69	35	50.7	[39.2, 62.2]
Cross-field	600	270	45.0	[41.1, 49.0]
AI-facing	260	114	43.8	[37.9, 49.9]
Search	95	30	31.6	[23.1, 41.5]
Simple scalar	249	68	27.3	[22.2, 33.2]
Financial	496	62	12.5	[9.9, 15.7]

TABLE V

CONDITIONAL SV-SI RATE PER TARGET, ALL CONDITIONS AND MODEL CONDITIONS (D+E) ONLY, OVER DECIDED STRUCTURAL PASSES. CATEGORIES HOLD ONE TO FOUR TARGETS, SO A CATEGORY RATE CAN BE DRIVEN BY A SINGLE TARGET.

Target	Category	Decided	All (%)	LLM dec.	LLM (%)
F03	AI-facing	93	68.8	42	85.7
F35	AI-facing	82	7.3	37	2.7
F39	AI-facing	85	51.8	45	53.3
F07+F09	Cross-field	111	28.8	63	7.9
F15+F16	Cross-field	181	88.4	132	99.2
F20+F21	Cross-field	203	15.8	152	9.2
F31+F33	Cross-field	105	43.8	54	29.6
F12	Financial	109	17.4	58	19.0
F13	Financial	185	0.0	134	0.0
F14	Financial	117	34.2	66	21.2
F25	Financial	72	0.0	32	0.0
F28	Financial	13	23.1	0	–
F29	Free text	69	50.7	28	32.1
F18	Search	95	31.6	46	32.6
F01	Simple scalar	92	38.0	42	11.9
F04	Simple scalar	157	21.0	106	19.8
F36+F37	Structured	107	57.0	57	63.2

E. Ground-truth agreement

On judgement rules, the judge agreed with the human annotator on 70.6% of 126 rule verdicts where both gave a verdict (Cohen's $\kappa = 0.24$); the annotator was unsure on 3.6% of judgement-rule verdicts. Agreement is below the pre-registered 0.4, so every judgement-rule result in this paper is exploratory. The rule functions agreed with the annotator on 89.5% of objective-rule verdicts ($\kappa = 0.32$), and the

TABLE VI

PRE-DECLARED ONE-SIDED CONTRASTS ON THE CONDITIONAL SV-SI RATE (FISHER EXACT, HOLM-CORRECTED ACROSS THE THREE TESTS). EACH TEST IS ONE-SIDED IN THE PRE-REGISTERED DIRECTION (RATE 1 > RATE 2), SO AN ODDS RATIO BELOW 1 YIELDS p CLOSE TO 1. CONDITIONS: B RANDOM, E LLM WITH FIELD CONTEXT, G BENIGN-UNUSUAL (HUMAN-CURATED), D LLM WITHOUT FIELD CONTEXT. WITHIN-TARGET RESULTS ARE IN THE TEXT.

	Contrast	Rate 1 (%)	Rate 2 (%)	OR	p_{Holm}	Supported
H3a	E>B	24.5	53.3	0.28	1.00	no
H3b	E>G	24.5	17.2	1.56	0.14	no
H3c	D>B	37.7	53.3	0.53	1.00	no

TABLE VII

EXPLORATORY: SV-SI BY GENERATOR SIZE AND TEMPERATURE ON THE SAME 10 TARGETS, PROMPT FAMILIES P1–P4, CONDITIONS D AND E. THE OBJECTIVE-RULE RATE IS DECIDED FOR EVERY STRUCTURAL PASS AND SO IS COMPARABLE ACROSS ROWS; THE ALL-RULES RATE USES THE JUDGE, WHICH LABELLED 100 INPUTS PER E2B RUN AGAINST A 1,200-INPUT SAMPLE IN E1. p_{Holm} : TWO-SIDED FISHER TEST AGAINST THE E1 GENERATOR ($\dagger$ QWEN3:4B, $T = 0.8$, THE E1 REFERENCE), HOLM-CORRECTED ACROSS THREE COMPARISONS. OBJ.: SV-SI ON OBJECTIVE RULES AMONG STRUCTURAL PASSES. ALL: SV-SI AMONG DECIDED PASSES.

qwen3	Runs	n	Pass	Obj. (%)	p_H	All (%)	p_H
1.7b, $T=0.8$	1	340	334	16 (4.8)	0.001	48/137 (35.0)	0.02
4b, $T=0.8^\dagger$	3	878	798	9 (1.1)	ref.	73/327 (22.3)	ref.
8b, $T=0.8$	1	394	355	5 (1.4)	0.77	30/130 (23.1)	0.90
4b, $T=0$	1	274	252	7 (2.8)	0.15	26/99 (26.3)	0.84

annotator’s repeated annotations agreed with each other on 93.8 % ($\kappa = 0.71$). The objective-rule results below are decided by those rule functions rather than by the judge, but that single human check of them is itself weak, so they are better described as independent of the judge than as externally validated.

F. Detection quality (RQ5)

No design reached the target, so H5 is not supported (Table VIII). Detection is measured against the same labels, so the judgement-rule part of these results is exploratory as well. The objective-rule part is not. At the operating point fixed on the training folds, the best non-judge design, HYB, detected 27.1 % of SV-SI inputs (CI 21.3–32.5) at a held-out FPR of 5.6 %, which is above the 5 % it was tuned for. Most of what it catches comes from R. HYB and R both caught 100.0 % of the inputs that violate only objective rules, and 6.5 % and 0.0 % of those that violate a judgement rule. On all decided inputs the fast designs reach 33.3 % for R and 36.6 % for HYB. EMB on its own was no better than chance (AUROC 0.45).

The prompted designs rank inputs better, with AUROC 0.75 for SLM and 0.76 for JUDGE against 0.60 for HYB. Their scores take only a handful of values, so holding FPR to 5 % on the training fields forces the threshold to the top confidence level. There SLM detects 1.3 % and JUDGE 23.7 %. Taking each model’s own verdict without tuning, which is exploratory, SLM detects 69.1 % while flagging 20.1 % of valid inputs, and JUDGE detects 51.7 % while flagging 6.8 %. JUDGE

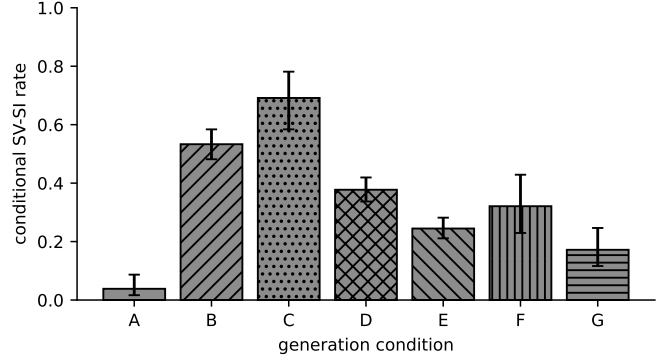


Fig. 1. Conditional SV-SI rate by generation condition with Wilson 95 % intervals, over decided structural passes (P5 excluded). C is the designed adversarial condition. A and G are the human-curated reference conditions for legitimate input.

gave no usable verdict for 26 % of inputs. These designs receive a rubric similar to the ground-truth judge’s, so their agreement with it partly measures agreement between two LLMs. Thresholds did not transfer across fields, and the held-out FPR per fold reached 30 % for the embedding-based designs. On benign-unusual inputs the FPR at the operating point was 4.8 % for HYB, 1.6 % for SLM and 1.6 % for JUDGE.

G. Overhead (RQ6)

Each configuration ran against the production build with only its own model loaded. No request failed and no layer call failed in any run (Table IX). H6 is not supported, because its first part fails. R added at most 1 ms to p97.5 latency with one client and 14 ms with eight, which is within the 50 ms bound. The negative differences at eight clients reflect run-to-run variation. EMB added 65–96 ms with one client, above the bound, and 122–751 ms with eight, because every request embeds its text on the same CPU. On the p95 bracket used as a sensitivity check, EMB fails on the review endpoint and is undetermined on the other two. The second part of H6 holds. SLM added 527–534 ms at the median with one client and JUDGE added 1611–7704 ms. The model server answers one request at a time, so requests queue under concurrency. With eight clients SLM added 4149–4210 ms at the median, and JUDGE completed no request within the 60 s window in all three runs on the course endpoint and in two of three on the review endpoint. Throughput on the course endpoint fell from 47.4 requests per second with structural validation alone to 1.8 with SLM and 0.1 with JUDGE. Set against detection quality (Fig. 2), only R and HYB lie on the trade-off frontier. SLM, JUDGE and EMB each cost more and detect less than one of them at the pre-registered operating point.

VIII. DISCUSSION

Throughout this section, every statement that rests on judgement rules is exploratory, because judge–human agreement fell below the pre-registered threshold. Statements about objective rules do not depend on the judge.

TABLE VIII

DETECTION QUALITY OF THE CANDIDATE SEMANTIC VALIDATION LAYERS ON THE SHARED STRATIFIED SAMPLE OF 575 DECIDED INPUTS (236 SV-SI, 339 VALID), LEAVE-FIELDS-OUT (5 FOLDS). RECALL, 95% BOOTSTRAP INTERVAL, FPR AND PRECISION ARE AT THE PRE-REGISTERED OPERATING POINT: THE THRESHOLD IS THE SMALLEST SCORE WHOSE FALSE-POSITIVE RATE ON VALID TRAINING-FOLD INPUTS IS AT MOST 5%, APPLIED TO HELD-OUT FIELDS. AUROC IS THRESHOLD-FREE. THE NATIVE COLUMN (EXPLORATORY, NOT PRE-REGISTERED) APPLIES EACH DESIGN’S OWN DECISION RULE WITHOUT TUNING: FOR SLM AND JUDGE, THE MODEL ANSWERED “NOT APPROPRIATE”; FOR R, EMB AND HYB, SCORE ≥ 0.5 . JUDGE RETURNED NO USABLE VERDICT FOR 26% OF INPUTS, COUNTED AS NOT FLAGGED. R’S ZERO FPR IS PARTLY BY CONSTRUCTION: ITS RULE FUNCTIONS ALSO DECIDE THE GROUND TRUTH FOR OBJECTIVE RULES. JUDGE SHARES A MODEL FAMILY WITH THE GENERATOR AND IS A REFERENCE POINT. LATENCY IS THE MEDIAN DECISION TIME OF THE LAYER ALONE ON THE STUDY CPU; HYB’S FIGURE REUSES EMBEDDINGS ALREADY CACHED BY EMB, SO ITS UNCACHED REQUEST-PATH COST IS THE ONE IN TABLE IX.

Design	Recall (%)	95% CI	FPR (%)	Precision (%)	AUROC	Native recall / FPR (%)	Median (ms)
HYB	27.1	[21.3, 32.5]	5.6	77.1	0.60	24.2 / 2.1	0.01
JUDGE	23.7	[18.5, 29.4]	3.5	82.4	0.76	51.7 / 6.8	2709
R	22.5	[17.0, 27.8]	0.0	100.0	0.61	22.5 / 0.0	0.01
EMB	4.7	[2.2, 7.3]	5.6	36.7	0.45	1.7 / 2.1	31
SLM	1.3	[0.0, 2.9]	1.2	42.9	0.75	69.1 / 20.1	1075

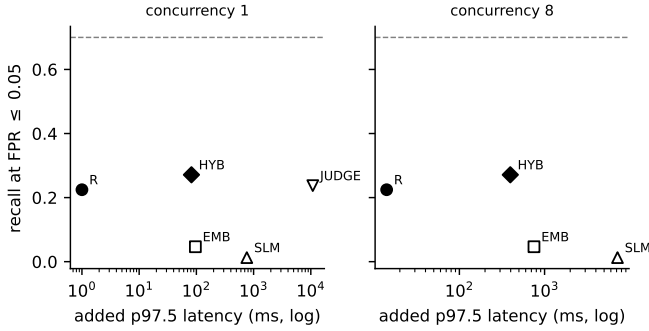


Fig. 2. Detection versus request-path cost for each semantic layer: recall at FPR ≤ 0.05 on held-out fields (Table VIII) against the largest added p97.5 latency over the three endpoints (Table IX), per concurrency level. Filled markers lie on the trade-off frontier. The dashed line is the pre-registered 70% recall target. JUDGE at concurrency 8 saturated and is drawn at its lower bound.

RQ1: the gap is measurable, and its size depends on what is asked. About a third of structurally valid LLM inputs broke an unenforced rule. The figure is a fifth when the model was asked only for legitimate values, and under a tenth on objective rules alone. This extends output-side findings [11], [13], [15] to inputs that an application would store. The magnitude is sensitive to prompt intent, to one date-dependent target, and to the judge, whose agreement with a human annotator was $\kappa = 0.24$. The claim that survives every check is that the gap is above the 5% bound, not any single figure.

RQ2: where it occurs is uneven. Category is moderately associated with the outcome, but individual targets drive that association. One target with a clock-dependent rule accounts for over half of the cross-field violations, and the structured category is a single target that the judge scored without its context.

RQ3: the gap is not specific to AI. This is the central negative result. Random values that satisfy the schema broke rules more often than model values, with or without field context, both pooled and within target. The results point to the validation boundary rather than to the producer. A schema

admits many values the application should reject, and any producer that satisfies the schema without knowing the rules will supply some of them. What distinguishes the model is that context helps. Giving it the purpose and the rules cut the odds of a violation by 77% relative to the schema alone. The random generator produces word sequences with no attempt at meaning, so it is a weak producer, and the comparison bounds non-AI input rather than characterising it.

RQ4: most violations are ordinary. The single most frequent violation was of an objective rule that the application could enforce directly, so enforcing objective rules would remove part of the gap. Judgement-rule violations are still the majority. On AI-facing fields most LLM violations came from ordinary prompts rather than from injection attempts, so the gap is mostly unenforced business logic rather than attack.

RQ5: no cheap layer closes it. The fast designs that can be held to a low FPR, R and HYB, find about a quarter of the violations at the operating point, and most of what they find are objective violations that could simply be enforced. JUDGE at a similar FPR also finds about a quarter, split evenly between objective and judgement rules. The prompted models discriminate better but produce coarse, field-dependent scores. Used as they are, they catch more while rejecting between one valid input in fifteen and one in five in this sample. Detecting judgement-rule violations remains the open problem.

RQ6: cost rises faster than detection. On commodity CPU hardware only the rule layer is effectively free, and it detects violations of rules the application could have enforced in the first place. The embedding layer costs tens of milliseconds per request and detected almost nothing here. The prompted models cost half a second to several seconds per request with one client, and a local model server handles one request at a time, so they cannot keep up with eight. Their better ranking of inputs does not survive the false-positive constraint. On this hardware the practical recommendation is to enforce objective rules in code and to treat judgement rules as an open problem, rather than to add a model to the request path.

Alternative explanations. The six explanations fixed in advance fare as follows. *Weak business rules rather than AI:* consistent with the data, because the gap appears for every

TABLE IX

REQUEST-PATH OVERHEAD OF EACH CONFIGURATION (PRODUCTION BUILD, 4-CORE CPU, NO GPU). EACH CELL IS THE MEDIAN OVER 3 REPETITIONS OF A 60 S AUTOCANNON RUN AFTER A 10 S WARM-UP, WITH A UNIQUE PAYLOAD PER REQUEST. ADDED LATENCY IS RELATIVE TO STRUCTURAL-ONLY VALIDATION ON THE SAME ENDPOINT AND CONCURRENCY. H6 IS EVALUATED ON THE ADDED P97.5, THE NEAREST TAIL PERCENTILE AUTOCANNON REPORTS ($\geq$ P95). LAYER SUCCESS IS THE SHARE OF LAYER CALLS THAT RETURNED A VERDICT RATHER THAN FAILING OPEN. “>60 000”: THE CONFIGURATION SATURATED, COMPLETING NO REQUEST WITHIN AT LEAST TWO OF THE THREE 60 S RUNS.

Config.	Endpoint	p50 (ms)	p97.5 (ms)	Δ p50 (ms)	Δ p97.5 (ms)	req/s	Layer ok (%)	H6
<i>Concurrency 1</i>								
structural	courses	20	28	0	0	47.4	–	
R	courses	20	29	0	1	46.0	100	holds
EMB	courses	47	93	27	65	19.2	100	fails
HYB	courses	46	93	26	65	19.2	100	
SLM	courses	547	789	527	761	1.8	100	
JUDGE	courses	7185	10214	7165	10186	0.1	100	holds
structural	profile	21	32	0	0	42.9	–	
R	profile	22	33	1	1	41.9	100	holds
EMB	profile	57	102	36	70	15.9	100	fails
HYB	profile	58	100	37	68	15.6	100	
SLM	profile	552	687	531	655	1.8	100	
JUDGE	profile	1632	7715	1611	7683	0.5	100	holds
structural	reviews	20	31	0	0	45.6	–	
R	reviews	22	31	2	0	43.1	100	holds
EMB	reviews	68	127	48	96	13.2	100	fails
HYB	reviews	65	113	45	82	13.6	100	
SLM	reviews	554	729	534	698	1.7	100	
JUDGE	reviews	7724	10855	7704	10824	0.1	100	holds
<i>Concurrency 8</i>								
structural	courses	146	322	0	0	48.3	–	
R	courses	149	317	3	-5	47.6	100	holds
EMB	courses	272	444	126	122	27.9	100	fails
HYB	courses	249	453	103	131	29.0	100	
SLM	courses	4295	7178	4149	6856	1.7	100	
JUDGE	courses	>60 000	>60 000	>60 000	>60 000	0.0	100	holds
structural	profile	156	477	0	0	41.7	–	
R	profile	158	341	2	-136	44.8	100	holds
EMB	profile	355	787	199	310	20.6	100	fails
HYB	profile	325	752	169	275	22.9	100	
SLM	profile	4354	7257	4198	6780	1.7	100	
JUDGE	profile	14438	23951	14282	23474	0.4	100	holds
structural	reviews	155	333	0	0	45.6	–	
R	reviews	161	347	6	14	43.9	100	holds
EMB	reviews	443	1084	288	751	16.0	100	fails
HYB	reviews	425	729	270	396	17.7	100	
SLM	reviews	4365	7513	4210	7180	1.7	100	
JUDGE	reviews	>60 000	>60 000	>60 000	>60 000	0.0	100	holds

producer that does not know the rules. *Random inputs produce the same effect*: consistent, and random inputs exceed model inputs. *Unusual human inputs behave the same*: not resolved. Human-curated unusual inputs did not differ detectably from context-aware model inputs (1.32, 0.77–2.27), but the interval spans both directions and the contrast is exploratory. *Confined to free text*: not supported, because structured and cross-field targets also show high rates, although single targets drive them. *Prompt injection under another name*: not supported. Injection prompts are excluded from all rates and account for 20 of 81 LLM violations on AI-facing fields. *Too expensive to deploy*: supported for the model-based layers on CPU hardware, and not supported for rules.

IX. THREATS TO VALIDITY

Internal validity. Labelling is the main threat. Judgement rules were decided by one 4B judge (agreement with a human annotator $\kappa = 0.24$ on 126 shared verdicts, below the pre-

registered 0.4, so judgement-rule results are exploratory), and seven of them were decided without the fields they depend on. Excluding those seven lowers the LLM rate to 24.5 % and lowers the random rate by more (Table III), which leaves the RQ3 direction intact. Agreement on semantic judgements can be low even between humans [72]. The disagreement here is not one-sided noise. On the shared verdicts the judge called 37 violations against the annotator’s 28, so the judge is the stricter rater and judgement-rule rates may be too high. The judge also returned no usable verdict on 36 further rules that the annotator decided, and the agreement statistic cannot use those. The annotator in turn disagreed with the deterministic rule functions on objective rules ($\kappa = 0.32$, 89.5 % agreement, 19 rule-function violations against 4), and those functions decide such rules from the record and the clock. Part of the low agreement is therefore annotation error, or the limited context the annotator was shown, rather than judge error alone. The annotator was self-consistent on the repeated items ($\kappa = 0.71$).

The judge sample gives the conditions different target mixes, and target-weighted rates are close to the pooled ones. Values generated in one call are correlated, so the intervals understate uncertainty. Time-relative rules are evaluated against the clock at the moment the labels are merged, so a later re-run can change labels near the boundary. The judge ran once, and its run-to-run stability on this CPU was not measured.

Defects found and corrected. Six defects were found during execution, and each is recorded as a deviation with the invalid data preserved. (i) The database was not reset between booking measurements, so course capacity ran out. Bookings are now reset before each measurement and the booking targets were re-measured. (ii) A resume routine retrieved failed generation calls, and the retrieved records are excluded. (iii) Model outputs with other key names were silently replaced by form defaults, and those records are excluded as generation failures. (iv) A first overhead run never enabled the layer, and it is discarded. (v) The first detector analysis selected thresholds with a quantile, which fails for coarse scores, and it was replaced before any detector result was reported. (vi) The label merge never passed human verdicts to the ground-truth algorithm, and the label importer dropped cells after an empty one. Both were found and fixed before the human annotation was collected, and the annotation was imported with the corrected code. Defects (iii), (v) and (vi) were found by independent review of the analysis and the code.

External validity. The study covers one application in one domain, built by the researcher, with one enforcement profile, so the results may not transfer. Generation uses quantised open models of 1.7 to 8 billion parameters on CPU. Frontier models may behave differently, and the evidence that scale does not predict semantic quality [13], [15] cuts both ways. The prompts never state the current date, which inflates date-rule violations for model inputs.

Construct validity. Semantic invalidity is defined by a frozen reference that another team would write differently. The reference was frozen before any input existed, and violations are reported per rule, so a reader can discount rules they disagree with.

Conclusion validity. Pooling three runs inflates the LLM sample, and per-run rates are reported for that reason. The category mapping assigns dual-tagged fields to one category, which was decided before the data existed but not logged at the time. The three contrasts H3a to H3c were the only pre-declared ones, but they are computed on labels that include judgement rules, so they too are exploratory under the pre-registered agreement gate, and only results decided by the rule functions alone remain confirmatory.

Who produced the human data. I carried out both the review and the annotation, and I also built the subject application and wrote the rule reference. The annotation is blind to the generation condition, because the interface never shows it, but it is not independent of the study, and a single annotator cannot establish how far the disagreement with the judge generalises. The review took 19 minutes for 272 values, a median of 1 s per value, and the annotation took 33 minutes for 200 items, a

median of 3 s per item. Both are well below the effort budgeted for them, which may itself contribute to the low agreement. The per-item answers and their timestamps are released with the artefacts.

Provenance. The reference conditions A and G were specified as human-written and their first drafts came from a language model. I then reviewed and curated every value, so H3b compares model input with human-curated input rather than with input written unaided. H3b remains exploratory because it depends on judgement rules.

X. CONCLUSION AND FUTURE WORK

Schema validation answers whether an input is well formed, not whether it belongs. At the input boundary of one application with an explicit rule reference, about a third of structurally valid LLM inputs broke an unenforced business rule. Random structurally valid inputs broke rules more often, and telling the model the field’s purpose and rules reduced violations. The gap is therefore a property of what the application leaves unenforced and is not specific to AI-generated input. The single most frequent violation was of an objective rule that a line of validation code could enforce. None of the five candidate layers detected the pre-registered share of violations at a tolerable false-positive rate. A human annotation of 200 inputs agreed with the judge only at $\kappa = 0.24$, below the threshold fixed in advance, so every result that depends on judgement rules is exploratory, and the ground truth for those rules is the weakest part of the study. On a four-core CPU, rules added at most 14 ms at p97.5 and embeddings tens of milliseconds. At the median a small language model added about half a second and an LLM judge several seconds, and the judge saturated on two of three endpoints with eight concurrent clients.

Future work should strengthen the ground truth for judgement rules with several annotators and more time per item, collect reference inputs written by several people unaided, re-judge the context-dependent rules with the full record, state the current date to the generators, and repeat the protocol on an existing open-source application.

ACKNOWLEDGMENT

The testbed, the harness, the rule reference, every generated input, measurement and label, and the scripts that produce every table, figure and number in this paper are available at <https://github.com/zahidrahimoon/semantic-validation-gap>. Language models were used as instruments of the study and as writing and coding assistants during its preparation; the design, the analysis and the claims are mine, and I take responsibility for them.

REFERENCES

- [1] F. Pezoa, J. L. Reutter, F. Suarez, M. Ugarte, and D. Vrgoč, “Foundations of JSON schema,” in *Proceedings of the 25th International Conference on World Wide Web*, ser. WWW ’16. International World Wide Web Conferences Steering Committee, Apr. 2016, pp. 263–273. [Online]. Available: <http://dx.doi.org/10.1145/2872427.2883029>

- [2] L. Attouche, M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, "Validation of modern JSON schema: Formalization and complexity," *Proceedings of the ACM on Programming Languages*, vol. 8, no. POPL, pp. 1451–1481, 2024. [Online]. Available: <https://doi.org/10.1145/3632891>
- [3] M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, *An Empirical Study on the "Usage of Not" in Real-World JSON Schema Documents*. Springer International Publishing, 2021, pp. 102–112. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-89022-3_9
- [4] V. Felmetzger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium*. USENIX Association, 2010. [Online]. Available: https://www.usenix.org/legacy/event/sec10/tech/full_papers/Felmetzger.pdf
- [5] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings 2014 Network and Distributed System Security Symposium*, ser. NDSS 2014. Internet Society, 2014. [Online]. Available: <http://dx.doi.org/10.14722/ndss.2014.23021>
- [6] P. Bisht, T. Hinrichs, N. Skrupsky, R. Bobrowicz, and V. N. Venkatakrishnan, "NoTamper: automatic blackbox detection of parameter tampering opportunities in web applications," in *Proceedings of the 17th ACM conference on Computer and communications security*, ser. CCS '10. ACM, Oct. 2010, pp. 607–618. [Online]. Available: <http://dx.doi.org/10.1145/1866307.1866375>
- [7] D. Corradini, M. Pasqua, and M. Ceccato, "Automated black-box testing of mass assignment vulnerabilities in RESTful APIs," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, May 2023, pp. 2553–2564. [Online]. Available: <http://dx.doi.org/10.1109/icse48619.2023.00213>
- [8] T. Li, C. Cui, R. Huang, D. Towey, and L. Ma, "Large language models for automated web-form-test generation: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 3, pp. 1–37, Feb. 2026. [Online]. Available: <http://dx.doi.org/10.1145/3735553>
- [9] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah, "Semantic constraint inference for web form test generation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA '24. ACM, Sep. 2024, pp. 932–944, arXiv:2402.00950. [Online]. Available: <http://dx.doi.org/10.1145/3650212.3680332>
- [10] B. Li, Y. Wang, H. Fei, J. Li, W. Ji, M.-L. Lee, and W. Hsu, "FormFactory: An interactive benchmarking suite for multimodal form-filling agents," in *Proceedings of the 33rd ACM International Conference on Multimedia*. ACM, 2025, pp. 13 273–13 280. [Online]. Available: <https://doi.org/10.1145/3746027.3758285>
- [11] Y. Li, "When JSON is not enough: Semantic reliability of schema-constrained LLM ordering agents," arXiv preprint arXiv:2607.18261, 2026. [Online]. Available: <https://arxiv.org/abs/2607.18261>
- [12] J. Ray, "The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models," arXiv preprint arXiv:2605.26128, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26128>
- [13] A. K. Singh, H. V. Khurdula, Y. D. Khemlani, and V. Agarwal, "The structured output benchmark: A multi-source benchmark for evaluating structured output quality in large language models," arXiv preprint arXiv:2604.25359, 2026. [Online]. Available: <https://arxiv.org/abs/2604.25359>
- [14] A. Sigdel and R. Baral, "Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance," arXiv preprint arXiv:2603.13404, 2026. [Online]. Available: <https://arxiv.org/abs/2603.13404>
- [15] X. Wrenn, R. Raykov, A. Angelov, H. Kitahara, Y. Watanabe, and A. Sailer, "Evaluating LLM trade-offs for enterprise automation: Lessons from workflow generation in a production enterprise platform," arXiv preprint arXiv:2608.03311, 2026. [Online]. Available: <https://arxiv.org/abs/2608.03311>
- [16] H. G. Tigulla and M. Vieira, "LLM-based robustness testing of microservice applications: An empirical study," arXiv preprint arXiv:2605.14202, 2026. [Online]. Available: <https://arxiv.org/abs/2605.14202>
- [17] M. Kim, S. Sinha, and A. Orso, "LlamaRestTest: Effective REST API testing with small language models," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 465–488, Jun. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3715737>
- [18] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '25. ACM, Jun. 2024, pp. 341–352, arXiv:2501.15145. [Online]. Available: <http://dx.doi.org/10.1145/3714393.3726501>
- [19] I. Ahmed, A. A. Borsha, R. D. Prangon, A. fuad Ahmad, and T. H. Tran, "Reflex-guard: A low-latency guardrail for LLM prompt safety using dense semantic embeddings," arXiv preprint arXiv:2608.17556, 2026. [Online]. Available: <https://arxiv.org/abs/2608.17556>
- [20] G. Saleem, N. Ahmed, M. I. Zaman, A. Hassan, and U. Mujahid, "A layered security framework against prompt injection in RAG-based chatbots," arXiv preprint arXiv:2606.19660, 2026. [Online]. Available: <https://arxiv.org/abs/2606.19660>
- [21] X. Wang, Z. Ji, W. Wang, Z. Li, D. Wu, and S. Wang, "SoK: Evaluating jailbreak guardrails for large language models," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 39–58. [Online]. Available: <https://doi.org/10.1109/SP63933.2026.00076>
- [22] OWASP Cheat Sheet Series Team, "Input validation cheat sheet," OWASP Cheat Sheet Series, OWASP Foundation, 2026, accessed: Sep; 22, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- [23] B. Metin, M. Wynn, A. Tunali, and Y. Kepir, "Business logic vulnerabilities in the digital era: A detection framework using artificial intelligence," *Information*, vol. 16, no. 7, p. 585, Jul. 2025. [Online]. Available: <http://dx.doi.org/10.3390/info16070585>
- [24] A. Martin-Lopez, S. Segura, C. Muller, and A. Ruiz-Cortes, "Specification and automated analysis of inter-parameter dependencies in web APIs," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2342–2355, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1109/tsc.2021.3050610>
- [25] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, *A Catalogue of Inter-parameter Dependencies in RESTful Web APIs*. Springer International Publishing, 2019, pp. 399–414. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-33702-5_31
- [26] S. Hanna and M. Munro, "Test case generation for semantic-based user input validation of web applications," *International Journal of Web Engineering and Technology*, vol. 13, no. 3, p. 225, 2018. [Online]. Available: <http://dx.doi.org/10.1504/ijwet.2018.095186>
- [27] N. Singh, A. Abuadba, Y. Gao, S. Nepal, and H. Kim, "Shifting from injection to interaction: Rethinking web security in the age of LLMs and beyond," arXiv preprint arXiv:2609.03999, 2026. [Online]. Available: <https://arxiv.org/abs/2609.03999>
- [28] S. Barakat, A. B. Sánchez, and S. Segura, *IDLGen: Automated Code Generation for Inter-parameter Dependencies in Web APIs*. Springer Nature Switzerland, 2023, pp. 153–168. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-48421-6_11
- [29] Z. Hatfield-Dodds and D. Dygalo, "Deriving semantics-aware fuzzers from web API schemas," arXiv preprint arXiv:2112.10328, 2021. [Online]. Available: <https://arxiv.org/abs/2112.10328>
- [30] F. Faisal Fadlalla and H. T. Elshoush, "Input validation vulnerabilities in web applications: Systematic review, classification, and analysis of the current state-of-the-art," *IEEE Access*, vol. 11, pp. 40 128–40 161, 2023. [Online]. Available: <http://dx.doi.org/10.1109/access.2023.3266385>
- [31] L. Pan, S. Cohnney, T. Murray, and V.-T. Pham, "EDEFuzz: A web API fuzzer for excessive data exposures," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. ACM, Feb. 2024, pp. 1–12. [Online]. Available: <http://dx.doi.org/10.1145/3597503.3608133>
- [32] O. Raz, P. Koopman, and M. Shaw, "Semantic anomaly detection in online data sources," in *Proceedings of the 24th international conference on Software engineering - ICSE '02*, ser. ICSE '02. ACM Press, 2002, p. 302. [Online]. Available: <http://dx.doi.org/10.1145/581376.581378>
- [33] M. Armillotta, N. Romandini, R. Montanari, and L. Cavallaro, "Antaeus: Hunting repository-level logic vulnerabilities via context-grounded LLM reasoning," arXiv preprint arXiv:2607.01138, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01138>
- [34] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in software security: A survey of vulnerability detection techniques and insights," *ACM Computing Surveys*, vol. 58, no. 5, pp. 1–35, Nov. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3769082>

- [35] M. Kim, T. Stennett, D. Shah, S. Sinha, and A. Orso, "Leveraging large language models to improve REST API testing," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER'24. ACM, Apr. 2024, pp. 37–41. [Online]. Available: <http://dx.doi.org/10.1145/3639476.3639769>
- [36] H. Huynh, Q.-T. Le, T. N. Nguyen, and V. Nguyen, "Using LLM for mining and testing constraints in API testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. ACM, Oct. 2024, pp. 2486–2487. [Online]. Available: <http://dx.doi.org/10.1145/3691620.3695341>
- [37] C. Cui, T. Li, J. Wang, C. Chen, D. Towey, and R. Huang, "Large language models for mobile gui text input generation: An empirical study," arXiv preprint arXiv:2404.08948, 2024. [Online]. Available: <https://arxiv.org/abs/2404.08948>
- [38] S. Mishra, "Reliability-aware structured synthetic data generation via schema enforcement and layered repair," in *Proceedings of the Workshop on Synthetic Data Generation and Management for Building AI Systems*, ser. SynthAI '26. ACM, May 2026, pp. 7–12. [Online]. Available: <http://dx.doi.org/10.1145/3814574.3816747>
- [39] Y. Song, P. Rajput, T. Sun, S. Ezzini, T. F. Bissyandé, and J. Klein, "Empirical study for structured output control in LLMs for software engineering," arXiv preprint arXiv:2606.09395, 2026. [Online]. Available: <https://arxiv.org/abs/2606.09395>
- [40] J. Zhao, M. Guan, and D. Liu, "Constraint-aware synthetic tabular data generation via inter-column constraint discovery with LLM agents," arXiv preprint arXiv:2608.15109, 2026. [Online]. Available: <https://arxiv.org/abs/2608.15109>
- [41] P. Sigloch and C. Benzmler, "Neuro-symbolic verification of LLM outputs for data-sensitive domains (extended preprint)," arXiv preprint arXiv:2605.26942, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26942>
- [42] P. Yu, W. Liu, Y. Yang, J. Li, Z. Zhang, X. Feng, and F. Zhang, "Benchmarking LLM tool-use in the wild," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, arXiv:2604.06185. [Online]. Available: <https://openreview.net/forum?id=y7fL5vfpn>
- [43] G. Yu, X. Sun, Z. Huang, S. Fan, Z. Zhang, X. Hu, X. Xue, Y. Shi, X. Xiao, Y. Song, B. Lyu, R. Wen, X. Li, Q. He, S. Zhu, and Z. Liu, "Getting the parameters right: A difficulty-graded benchmark and probe-guided training for LLM tool calls," arXiv preprint arXiv:2608.03071, 2026. [Online]. Available: <https://arxiv.org/abs/2608.03071>
- [44] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Machine Learning and Systems (MLSys)*, vol. 1, 2019, pp. 334–347. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html
- [45] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.
- [46] M. Hulsebos, K. Hu, M. Bakker, E. Zraggen, A. Satyanarayan, T. Kraska, C. Demiralp, and C. Hidalgo, "Sherlock: A deep learning approach to semantic data type detection," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 1500–1508.
- [47] A. Banerjee and V. T. Subrahmaniam, "Deep learning based context embedding approach for detecting data entry errors," US Patent 12 288 159, Apr. 29, 2025. [Online]. Available: <https://patents.google.com/patent/US12288159B2>
- [48] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, ser. CCS '23. ACM, Nov. 2023, pp. 79–90, arXiv:2302.12173. [Online]. Available: <http://dx.doi.org/10.1145/3605764.3623985>
- [49] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt injection attack against LLM-integrated applications," arXiv preprint arXiv:2306.05499, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05499>
- [50] Y. Kaya, A. Landerer, S. Pletinckx, M. Zimmermann, C. Kruegel, and G. Vigna, "When AI meets the web: Prompt injection risks in third-party AI chatbot plugins," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 4223–4242. [Online]. Available: <https://doi.org/10.1109/SP63933.2026.00062>
- [51] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupe>
- [52] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, "Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, Apr. 2025, pp. 1768–1780, arXiv:2308.01990. [Online]. Available: <http://dx.doi.org/10.1109/icse55347.2025.00007>
- [53] S. Tsigkopoulos and C. Ntantogian, "From prompt injection to web exploitation: Revisiting classic vulnerabilities in LLM-integrated applications," arXiv preprint arXiv:2608.10281, 2026. [Online]. Available: <https://arxiv.org/abs/2608.10281>
- [54] S. Khodayari, X. Zhang, B. Acharya, and G. Pellegrino, "Indirect prompt injection in the wild: An empirical study of prevalence, techniques, and objectives," arXiv preprint arXiv:2604.27202, 2026. [Online]. Available: <https://arxiv.org/abs/2604.27202>
- [55] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD '25. ACM, Jul. 2025, pp. 1809–1820. [Online]. Available: <http://dx.doi.org/10.1145/3690624.3709179>
- [56] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in *Advances in Neural Information Processing Systems 38*. Neural Information Processing Systems Foundation, Inc., 2025, pp. 22 436–22 458. [Online]. Available: <https://doi.org/10.52202/085713-0666>
- [57] Y. Liu, R. Xu, X. Wang, Y. Jia, and N. Z. Gong, "WAIInjectBench: Benchmarking prompt injection detections for web agents," arXiv preprint arXiv:2510.01354, 2025. [Online]. Available: <https://arxiv.org/abs/2510.01354>
- [58] A. Milani, V. Franzoni, and E. Florindi, "Indirect prompt injection in large language models," *Neural Computing and Applications*, vol. 38, no. 13, Jun. 2026. [Online]. Available: <http://dx.doi.org/10.1007/s00521-026-12266-x>
- [59] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, May 2025, pp. 2190–2208. [Online]. Available: <http://dx.doi.org/10.1109/sp61157.2025.00250>
- [60] M. A. Ayub and S. Majumdar, "Embedding-based classifiers can detect prompt injection attacks," in *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024)*, ser. CEUR Workshop Proceedings, R. Allen, S. Samtani, E. Raff, and E. Rudd, Eds., vol. 3920. Arlington, Virginia, USA: CEUR-WS.org, 2024, pp. 257–268. [Online]. Available: <https://ceur-ws.org/Vol-3920/paper15.pdf>
- [61] Y. Datta and S. Rajasekar, "JavelinGuard: Low-cost transformer architectures for LLM security," arXiv preprint arXiv:2506.07330, 2025. [Online]. Available: <https://arxiv.org/abs/2506.07330>
- [62] K. Lakara, R. Shi, and F. Silavong, "Fence: Specialized SLM guardrails for LLM applications," arXiv preprint arXiv:2607.18268, 2026. [Online]. Available: <https://arxiv.org/abs/2607.18268>
- [63] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, B. Alomair, X. Zhao, W. Y. Wang, N. Gong, W. Guo, and D. Song, "PromptArmor: Simple yet effective prompt injection defenses," arXiv preprint arXiv:2507.15219, 2025. [Online]. Available: <https://arxiv.org/abs/2507.15219>
- [64] A. Akinrele and S. N. Gowda, "Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals," arXiv preprint arXiv:2605.26999, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26999>
- [65] H. X. Le, B. Goh, and Q. A. Tang, "Prompt attack detection with LLM-as-a-judge and mixture-of-models," arXiv preprint arXiv:2603.25176, 2026. [Online]. Available: <https://arxiv.org/abs/2603.25176>
- [66] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Y. Feng and E. Lefever, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 431–445. [Online]. Available: <https://aclanthology.org/2023.emnlp-demo/40/>

- [67] H. Li and X. Liu, “InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models,” arXiv preprint arXiv:2410.22770, 2024. [Online]. Available: <https://arxiv.org/abs/2410.22770>
- [68] G. Zizzo, G. Cornacchia, K. Fraser, M. Z. Hameed, A. Rawat, B. Buesser, M. Purcell, P.-Y. Chen, P. Sattigeri, and K. Varshney, “Adversarial prompt evaluation: Systematic benchmarking of guardrails against prompt input attacks on LLMs,” arXiv preprint arXiv:2502.15427, 2025, presented at the NeurIPS 2024 Safe Generative AI Workshop (non-archival). [Online]. Available: <https://arxiv.org/abs/2502.15427>
- [69] D. Kumar, N. A. Birur, T. Baswa, S. Agarwal, and P. Harshangi, “No free lunch with guardrails,” arXiv preprint arXiv:2504.00441, 2025. [Online]. Available: <https://arxiv.org/abs/2504.00441>
- [70] A. C. Maiorano, “Evaluating prompt injection defenses for educational LLM tutors: Security-usability-latency trade-offs,” arXiv preprint arXiv:2605.06669, 2026. [Online]. Available: <https://arxiv.org/abs/2605.06669>
- [71] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, “Optimization-based prompt injection attack to LLM-as-a-judge,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. ACM, 2024, pp. 660–674. [Online]. Available: <https://doi.org/10.1145/3658644.3690291>
- [72] T. Calò, A.-E. Gurita, and L. De Russis, “Measuring the semantic accessibility gap in LLM-generated web UIs,” in *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA ’26. ACM, Apr. 2026, pp. 1–5. [Online]. Available: <http://dx.doi.org/10.1145/3772363.3799364>